
Syllabus for Real-Time Operating Systems

PGCP-ESD February 2026 Batch

Duration: 90 hours (20 theory hours + 40 lab hours + 30 hours self Learning)

Objective: To introduce students to real-time operating system concepts, principles, and development techniques for real-time embedded systems.

Prerequisites: Basic knowledge of operating systems and embedded systems.

Evaluation: 100 Marks (Theory exam – 40%, Lab exam – 40%, Internals – 20%)

Textbook:

- Real-Time concepts for Embedded Systems, Qing Li, Caroline Yao, CMP Media LLC

References:

- Real-Time Embedded Systems: Design Principles and Engineering Practices, Xiaocong Fan
- Real-Time Systems Design and Analysis: Tools for the Practitioner, 4th Edition, Philipp A. Laplante, Seppo J. Ovaska, Wiley Publisher
- The FreeRTOS Reference Manual, FreeRTOS online
- Use of FreeRTOS in Teaching Real-time Embedded Systems Design Course, Dr. Nannan He, Minnesota State University, Mankato
- RTOS Program Models Used in Embedded Systems, József Kopják, Dr. János Kovács

Session 1: Introduction to Real-Time Systems (2T + 4L + 3SL)

Lecture:

- What are Real-Time Systems?
- Characteristics of Real-Time Systems: Timing Constraints, Determinism, Predictability
- Types of Real-Time Systems: Hard Real-Time, Soft Real-Time
- Challenges in Real-Time System Design

Lab:

- Setting up Development Environment for Real-Time System Development
- Introduction to Real-Time Kernel Development Tools and APIs

Session 2,3: Real-Time Operating System Architecture (4T + 8L + 6SL)

Lecture:

- Overview of Real-Time Operating System (RTOS) Components
- Task Management: Task States, Task Scheduling, Context Switching
- Inter-Task Communication and Synchronization
- Memory Management and Resource Allocation

Lab:

- Exploring Task Management and Scheduling in Real-Time Kernels
- Implementing Inter-Task Communication Mechanisms

Session 4,5: Real-Time System Design Considerations (4T + 8L + 6SL)**Lecture:**

- Task Partitioning and Priority Assignment
- Deadline Analysis and Task Timing Requirements
- Resource Management and Allocation
- Fault Tolerance and Error Handling

Lab:

- Designing and Implementing Real-Time Systems with Task Partitioning and Priority Assignment
- Analyzing Task Timing Requirements and Resource Allocation

Session 6,7: Task Scheduling Algorithms (4T + 8L + 6SL)**Lecture:**

- Preemptive vs Non-Preemptive Scheduling
- Scheduling Policies: Rate Monotonic (RM), Earliest Deadline First (EDF), Fixed Priority, Dynamic Priority
- Analysis of Scheduling Algorithms: Response Time Analysis, Worst-Case Execution Time (WCET)

Lab:

- Implementing Various Task Scheduling Algorithms in Real-Time Systems
- Analyzing Scheduling Results and Performance Metrics

Session 8,9: Interrupt Handling in Real-Time Systems (4T + 8L + 6SL)**Lecture:**

- Introduction to Interrupts in Real-Time Systems
- Priority-Based Interrupt Handling
- Interrupt Latency and Response Time Analysis
- Interrupt Service Routines (ISRs) and Deferred Interrupt Handling

Lab:

- Configuring and Handling Interrupts in Real-Time Systems
- Analyzing Interrupt Latency and Response Time

Session 10: Inter-Process Communication (IPC) (2T + 4L + 3SL)

Lecture:

- Message Passing: Queues
- Shared Memory and Memory-Mapped I/O
- Semaphores, Mutexes, and Critical Sections

Lab:

- Implementing Message Passing Mechanisms for Inter-Process Communication
- Exploring Shared Memory and Synchronization Techniques

Self Learning Topics

1. Practical RTOS Internals (Deeper Insight)

- RTOS tick mechanism and tickless idle
- Context switch trigger sources
- Stack overflow detection techniques
- Idle task behavior and hooks

2. RTOS Memory Optimization

- Static vs dynamic task allocation
- Heap implementations in RTOS (conceptual)
- Stack size estimation per task
- Avoiding fragmentation in long-running systems

3. Priority Inversion – Advanced Handling

- Priority inheritance vs priority ceiling
- Real-world priority inversion examples
- Debugging priority inversion issues

4. RTOS Debugging Techniques

- Debugging deadlocks in RTOS
- Task starvation detection
- Analyzing missed deadlines
- RTOS trace tools (conceptual awareness)

5. Interrupt vs Task Design Decisions

- What belongs in ISR vs task
- Deferred interrupt handling strategies
- Latency-critical ISR design rules

6. Power-Aware RTOS Design

- Low-power tickless scheduling
- RTOS integration with sleep modes
- Power vs responsiveness trade-offs

7. RTOS Configuration & Portability

- Configuring RTOS for different MCUs
- RTOS porting basics (conceptual)
- Endianness and word-size considerations

8. Mixed-Criticality Systems

- Tasks with different criticality levels
- Isolation strategies
- Safety-critical vs non-critical tasks

9. RTOS and Drivers Interaction

- Driver execution context (ISR, task, deferred)
- Synchronization between drivers and application
- Buffering strategies

10. RTOS vs Bare-Metal Design

- When RTOS is overkill
- Bare-metal + super-loop vs RTOS
- Migration strategy from bare-metal to RTOS

11. RTOS Testing & Validation

- Unit testing RTOS-based code
- Stress testing with task overload
- Fault injection testing (conceptual)

12. Safety & Reliability Awareness

- Watchdog integration with RTOS
- Fail-safe task design
- Recovery after task failure

13. Industry RTOS Case Studies

- RTOS usage in automotive ECUs
- RTOS in medical devices
- RTOS in industrial control systems

14. Performance Tuning in RTOS

- Reducing context switch overhead
- Task execution time optimization
- Scheduling overhead minimization

15. Career & Interview Preparation

- Common RTOS interview pitfalls
- Explaining scheduling decisions
- Debug-based RTOS interview questions
- RTOS design trade-off discussions